

实验二：在线评测系统

全异步 FastAPI · Streamlit · Python/C++ 评测 · AI 智能命题

项目	内容
作者	liujh25
邮箱	liujh25@mails.tsinghua.edu.cn
完成日期	2026 年 9 月 6 日
版本	1.5.0

目录

- 1. 项目概述与目标
- 2. 系统架构与技术选型
- 3. 基础模块实现
- 4. AI 智能命题
- 5. 关键难点与安全设计
- 6. 测试与成果展示
- 7. AI 使用说明
- 8. 总结与改进建议

1. 项目概述与目标

本实验构建一个功能完整的小型 Online Judge。系统不仅实现题目、用户和提交的管理，还能够异步编译或解释用户程序，在时间与内存限制内逐测试点运行，并以统一 REST API 向 Streamlit 前端提供服务。进阶部分把可配置的大语言模型接入现有题目管理流程。

评分模块	完成内容	分值
Step 1	题目加载、校验、增删改查	5
Step 2	多语言、动态注册、资源限制	5
Step 3	列表、详情、重评	5
Step 4	Session、注册、登录、角色	5
Step 5	测试日志、公开策略、审计	5
Step 6	三组前端页面与 API 对接	5
Advance	R1-R4 与命题质量增强	10

2. 系统架构与技术选型

系统采用前后端分离结构。Streamlit 保存 httpx.Client 以复用 Cookie；FastAPI 所有业务路由均使用 async def；SQLAlchemy Async 通过 aiosqlite 持久化。评测和 AI 命题先保存 pending 记录，再使用 asyncio.create_task 在后台执行，客户端通过查询接口观察状态。

层次	技术	职责
交互层	Streamlit	用户、题目、提交和 AI 页面
协议层	FastAPI/Pydantic	异步 API、校验、统一错误
领域层	异步服务	权限、评测、日志、AI 工作流
数据层	SQLAlchemy Async/SQLite	业务数据与任务状态
执行层	asyncio subprocess/psutil	编译、运行、资源监控

Async OJ · OpenAPI 路由总览		
共 31 个 HTTP 操作，全部由 FastAPI 异步接口提供。		
POST	/api/auth/login	Login
POST	/api/auth/logout	Logout
POST	/api/users/	Register
GET	/api/users/	List Users
POST	/api/users/admin	Create Admin
GET	/api/users/{user_id}	Get User
PUT	/api/users/{user_id}/role	Update Role
GET	/api/problems/	List Problems
POST	/api/problems/	Add Problem
GET	/api/problems/{problem_id}	Get Problem
PUT	/api/problems/{problem_id}	Update Problem
DELETE	/api/problems/{problem_id}	Delete Problem
PUT	/api/problems/{problem_id}/log_visibility	Update Log Visibility
GET	/api/languages/	List Languages
POST	/api/languages/	Register Language
POST	/api/submissions/	Submit
GET	/api/submissions/	List Submissions
GET	/api/submissions/{submission_id}	Get Submission
PUT	/api/submissions/{submission_id}/rejudge	Rejudge
GET	/api/submissions/{submission_id}/log	Submission Log
GET	/api/logs/access/	Access Logs
POST	/api/reset/	Reset System
PUT	/api/ai/model-config	Set Model Config
GET	/api/ai/problem-tasks/	List Tasks
POST	/api/ai/problem-tasks/	Create Task
POST	/api/ai/luogu-problem-tasks/	Create Luogu Task
POST	/api/ai/problem-tasks/{task_id}/iterations	Iterate Task
POST	/api/ai/problem-tasks/{task_id}/test-refinements	Refine Testcases
GET	/api/ai/problem-tasks/{task_id}	Get Task
PUT	/api/ai/problem-tasks/{task_id}/cancel	Cancel Task
GET	/health	Health

图 1 FastAPI 自动生成的 OpenAPI 接口模式

3. 基础模块实现

3.1 题目与用户管理

题目模型强制验证 id、标题、题面、输入输出说明、样例、约束和测试点；可选字段统一补默认值。用户密码使用 bcrypt 哈希，随机 Session Token 仅以 SHA-256 摘要存入数据库，Cookie 设置 HttpOnly 与 SameSite。角色包括 user、admin 和 banned。

普通用户与管理员查看题目详情时都会得到 testcases。删除题目时，后端在同一事务中显式清理逐测试点结果、提交、访问审计和关联 AI 任务，避免不同 SQLite 外键配置造成残留数据。

做题模块先展示独立习题列表。固定高度区域可滚动浏览全部题目，关键词匹配题号、标题、题面、标签、难度、来源和作者，也可与标签组合。点击卡片进入评测工作区并可返回列表。



图 2 可滚动浏览与关键词查找的独立习题列表



图 3 题目、标签筛选与评测同屏工作区

3.2 异步评测与提交管理

系统内置 Python 和 C++14。动态语言命令先经模板校验，再拆分为参数数组交给 `create_subprocess_exec`，禁止 shell 管道、重定向和连接符。每次评测使用独立临时目录；超时或超内存时终止完整进程树。Linux 还设置地址空间、CPU、文件大小和进程数硬限制。

时间与内存限制分别独立按题目配置、语言配置、系统默认值的优先级解析，系统默认值为 3 秒和 128 MB。服务器安装 GCC 后，可通过页面或 `POST /api/languages/` 动态注册 C11。

结果	判定
AC	运行成功且规范化输出完全一致
WA	运行成功但输出不同
TLE/MLE	超过题目时间或内存限制
RE	程序非零退出
CE	编译失败
UNK	未分类的执行器异常

提交接口立即返回 `pending`。后台任务完成后写入总分、编译摘要、运行摘要和逐测试点日志。查询支持用户、题目、状态和分页；同一用户一分钟最多提交三次，管理员可原 ID 重新评测。

题目详情按 `problem_id` 查询提交：管理员可查看所有人在该题的记录，普通用户由后端强制收窄到本人。用户提交数仅统计该用户，满分通过数按不同题目去重，故单题最多贡献一次。

3.3 日志、审计与前端

测试点日志默认仅本人和管理员可见；管理员可以按题目公开。对存在的提交发起日志访问时，无论成功或因权限拒绝，都会以 `view_logs` 动作记录用户、题目、时间和状态，方便安全审计。前端所有操作经 REST API 完成，不能通过前端隐藏按钮绕过后端权限。

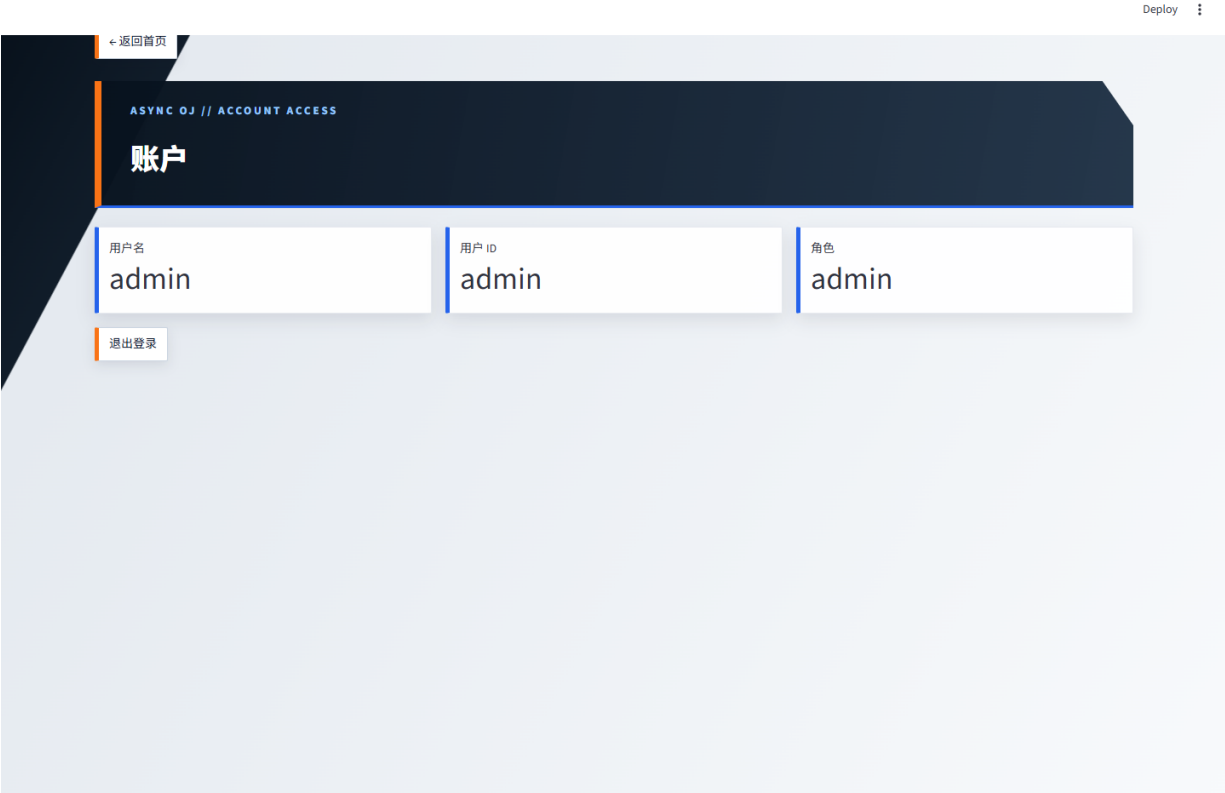


图 4 登录后的账户状态与服务端 Session

4. AI 智能命题

模型提供商 URL、模型名、API Key 和价格均由用户配置。API Key 使用运行时生成的 Fernet 主密钥加密落盘，不在响应、异常或日志中出现。系统兼容 OpenAI Chat Completions 接口，也提供本地假服务用于无密钥验收。

命题任务分为需求分析、草稿生成、边界测试复核、结构校验和完成导入。第二次模型调用专门检查题意一致性、边界条件、常见错误和复杂度区分度。最终 problem 必须通过与普通题目新增相同的模型校验，用户可一键导入表单继续人工审阅。



图 5 智能命题中的逐类型测试点配额

洛谷参考命题页接受 P1001 或 1001 形式的公开题号，并允许选择相似题或扩展题。服务端从固定 HTTPS 主机提取考点、难度、题面分区、样例与限制，再要求模型更换标题、叙事、数据设计和测试边界，生成可继续迭代的原创题目。外部题面始终按不可信数据处理，其中夹带的指令不会被执行；完整参考内容也不会再在任务 API 中回显。



图 6 洛谷题号、命题方式与测试点配额同屏配置

完成后的题目可以继续派生迭代版本：用户填写改进意见，系统把父版本完整结果与意见再次交给模型，并保留父任务和轮次。智能命题与迭代改进表单都以内嵌列表配置简单、普通、边界、时间限制、特殊情形、溢出、易错对抗与多样数据测试点，并可为每类单独指定数量。时间限制测试点要求给出确定输入输出并注明要淘汰的低效复杂度。



图 7 迭代意见与测试点配额同屏配置

系统分别累计输入和输出 Token，并按照用户配置的计价单位与单价计算美元费用。取消接口会调用 Task.cancel()，后台捕获 CancelledError 后把任务持久化为 cancelled，满足真实中断要求。

5. 关键难点与安全设计

难点	解决方案
异步状态一致性	先提交数据库事务，再调度后台任务；每个任务使用独立 AsyncSession。
跨平台进程控制	Windows 监控进程树；Linux 叠加 rlimit 和进程组终止。
错误优先级	认证/角色使用依赖先执行，统一将 Pydantic 422 转为规范的 400。
日志隐私	题目返回测试点；评测日志仍执行本人、管理员和公开策略。
模型密钥	加密存储、仅写接口、异常脱敏、禁止 URL 内嵌凭据。
外部题面	固定洛谷主机、题号白名单、超时/大小限制、提示注入隔离。

课程级评测器已经显著降低命令注入和资源失控风险，但它不是面向公网恶意代码的完整安全沙箱。生产部署仍需独立低权限工作节点、容器/虚拟机、seccomp、只读文件系统和网络隔离。

6. 测试与成果展示

自动测试共 29 项，覆盖用户与角色、认证优先级、题目 CRUD、限制继承、级联删除、测试点可见性、单题按角色提交范围、独立用户统计、动态 C 注册、语言命令校验、提交限流、筛选和重评、Python 的 AC/WA/RE/TLE、C++ CE、测试日志公开与审计、系统重置、AI 进度/取消/费用、洛谷页面解析与 Streamlit 浏览器冒烟。当前本机全部通过，后端语句覆盖率为 85.70%。

检查	结果
pytest	29 passed
Coverage	85.70% (门槛 80%)
Ruff	All checks passed
mypy	Success: no issues
CI	Ubuntu + Python 3.10/3.12 + GCC

6.1 页面验收

前端截图由 Playwright 在真实运行的 FastAPI 与 Streamlit 服务上自动采集。角色化首页显示等级、完成题数、AI 出题数与三层功能入口；管理员登录后用户管理入口自动解锁。截图证明前端可以独立启动并通过 Cookie Session 调用后端接口。



图 8 等级、资源统计与角色化功能入口

7. AI 使用说明

本项目使用 Codex 作为开发协作工具。工作流为：先逐页核对课程文档与 API 规范，形成决策完整的实施计划；随后按功能模块实现，每个重要里程碑执行语法或自动测试并创建 Conventional Commit；最后运行 Ruff、mypy、pytest、覆盖率和视觉报告检查。

AI 主要参与架构草拟、样板代码生成、测试用例扩展、错误定位与文档组织。所有外部课程要求均以课程文档为准，生成代码经过自动测试和人工结构审查。按新增源代码估算，Vibe Coding 比例约为 85%，其余为需求确认、验收设计和结果复核。

8. 总结与改进建议

实验完成了从 REST API、数据模型、异步任务、系统级进程控制到前端交互的完整闭环。最有价值的收获是区分提交任务状态与测试点判定，并通过持久化状态让异步流程具备可观察性。权限和审计也说明安全策略必须位于后端数据边界。

后续可将 SQLite 迁移至 PostgreSQL，引入 Redis/Celery 支持多工作节点；把评测进程迁移至无网络容器；用 SSE 推送提交和 AI 进度；为题目加入标准解与自动测试数据生成器，使 AI 测试点能够在隔离环境中被参考解自动验证。